# Nesne Yönelimli Programlama

## Hafta 5: Pointerlar ve Dinamik Bellek Yönetimi

Dr. Hüseyin Yüce

Marmara Üniversitesi Teknoloji Fakültesi

Mekatronik Mühendisliği Bölümü

# Hafta 5: Teorik ve Kavramsal Hedefler

Bu hafta hedeflenen temel kavramsal kazanımlar:

- **İşaretçi Mantığı ve Bellek Adresleme:** Adres ( `&` ) ve dolaylı erişim / deferans ( `*` ) operatörleri
- **İşaretçi Aritmetiği:** Adres artırma, azaltma, indisleme ve işaretçi farkı matematiği
- **Sabitler ve Göstericiler:** `const int*` , `int* const` ve `const int* const` farkları
- **Jenerik ve Çok Seviyeli Göstericiler:** `void*` , `static_cast` ve göstericiye gösterici ( `**` )
- **Bellek Anatomisi:** Yığın (*Stack*) ve Öbek (*Heap / Serbest Bellek*) mimarisi
- **Dinamik Bellek Yönetimi:** `new` , `delete` , dinamik diziler ( `new[]` , `delete[]` )
- **Kritik Bellek Hataları:** Bellek sızıntısı (*memory leak*), sarkan gösterici (*dangling pointer*) ve `nullptr` güvenliği

> **İşaretçi (Pointer):** Bellekte başka bir değişkenin veya veri yapısının fiziksel adresini tutan özel değişken.

# Hafta 5: Uygulamalı ve Mekatronik Hedefler

Laboratuvar oturumu ve gömülü sistem mimarisi kazanımları:

- Dinamik Telemetri Tamponu Tahsisi:
  - Mikrodenetleyici sensör kayıtları için `new[]` ve `delete[]` ile çalışma zamanında bellek yönetimi
- İşaretçi Aritmetiği ile Hızlı Veri Taraması:
  - Dizi indisleme ek yükü olmadan işaretçilerle sensör halka tamponu (*Ring Buffer*) gezintisi
- Fonksiyonlara Bellek Adresi Aktarımı:
  - Büyük telemetri bloklarının kopyalanmadan adres üzerinden hızlı aktarımı ve güncellenmesi
- Çok Boyutlu Dinamik Veri Yapıları:
  - 2B lidar tarama ve robot harita tablolarının `int**` gösterici dizisi ile tahsisi ve güvenli temizliği

**Ring Buffer (Halka Tampon):** Başı ile sonu birleşik, sabit boyutlu dairesel veri kuyruğu yapısı.

# Motivasyon: Neden Pointerlar ve Dinamik Bellek?

Mekatronik ve gömülü yazılımlarda donanıma doğrudan erişim ve bellek verimliliği esastır:

- **Donanım Seviyesinde Adresleme:** Mikrodenetleyici çevrebirim registerlarına (ADC, PWM, UART) doğrudan fiziksel bellek adresi üzerinden erişilir.
- **Yüksek Performans ve Sıfır Kopyalama:** Megabaytlarca görüntü veya nokta bulutu verisi fonksiyonlara kopyalanmadan yalnızca 8 baytlık adresle aktarılır ($\mathcal{O}(1)$ zaman).
- **Esnek Çalışma Zamanı Boyutlandırması:** Derleme anında kaç sensör verisi geleceği bilinmediğinde bellek ihtiyacı çalışma zamanında dinamik karşılanır.
- **Nesne Yönelimli Polimorfizmin Temeli:** İlerleyen haftalarda göreceğimiz çalışma zamanı çok biçimliliği (dinamik bağlama) taban sınıf işaretçileriyle yürütülür.

> **Mühendislik İlkesi:** İşaretçiler, yazılım ile bilgisayarın fiziksel donanımı ve RAM mimarisi arasındaki doğrudan köprüdür.

# Belleğin Temel Mimarisi ve Adresleme

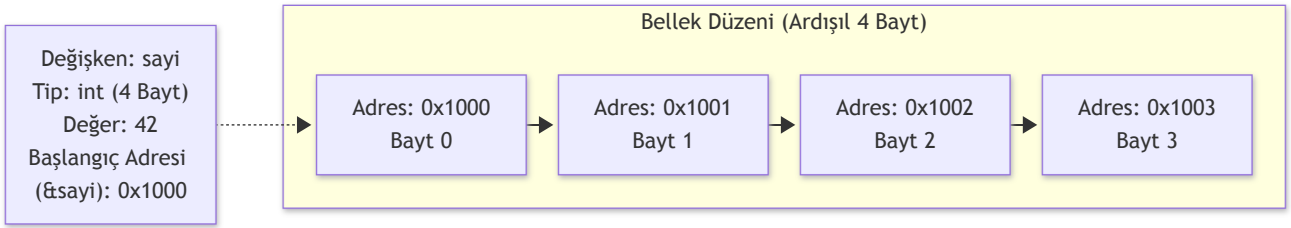
Bilgisayar belleği (RAM), her biri 1 bayt (8 bit) olan ardışıl hücrelerden oluşur:

- **Fiziksel Adres:** Her bellek baytının benzersiz bir onaltılık (*hexadecimal*) adresi bulunur (örn. `0x7ffee4ba5c08`).
- **Değişkenin 4 Temel Özelliği:**
  - **İsim:** Programcının koddaki erişim etiketi ( `sicaklik` )
  - **Tip:** Bellekte kaç bayt kapladığını ve bitlerin nasıl yorumlanacağını belirler ( `int` = 4 bayt)
  - **Değer:** Hücrelerin içerisindeki ikili (*binary*) veri ( `24` )
  - **Adres:** Değişkenin bellekte başladığı ilk baytın konumu ( `&sicaklik` )
- **İşaretçi Boyutu:** Bir işaretçinin kapladığı alan tuttuğu verinin tipine değil, işlemci mimarisine bağlıdır (32-bit sistemlerde 4 bayt, 64-bit sistemlerde 8 bayt).

> **RAM (Random Access Memory):** Herhangi bir hücresine eşit sürede doğrudan erişilebilen geçici ana bellek.

# Bellek Hücreleri ve Değişken Temsili

Bir `int sayi = 42;` değişkeninin 64-bit sistemdeki bellek anatomisi:




> **Adres Notu:** Çok baytlı veri tiplerinde değişkenin adresi ( `&değişken` ), verinin bellekte kapladığı ilk baytın (en düşük adresinin) adresidir.

# İşaretçi (Pointer) Nedir ve Nasıl Tanımlanır?

İşaretçi, değeri bir başka değişkenin bellek adresi olan özel bir değişkendir:

```
1   int sayi = 25;       // Standart tamsayı değişkeni
2   int* ptr = &sayi;    // sayi değişkeninin adresini tutan işaretçi
```

- Yıldız ( `*` ) Operatörü (Tanımda): Değişkenin bir işaretçi olduğunu belirtir ( `int* ptr;` veya `int *ptr;` ).
- Adres ( `&` ) Operatörü: Bir değişkenin bellek adresini elde etmek için kullanılır ( `&sayi` ).
- Dolaylı Erişim / Deferans ( `*` Operatörü - Kullanımda): İşaretçinin tuttuğu adrese giderek oradaki asıl değere erişir veya değeri günceller ( `*ptr = 30;` ).
- Tür Eşleşmesi Kuralı: İşaretçinin türü ile işaret ettiği değişkenin türü tam olarak eşleşmelidir ( `int*` yalnızca `int` adresini tutabilir).

Deferans (Dereference): İşaretçinin işaret ettiği adresteki gerçek veriye erişme işlemi.

# İşaretçi Tanımlama ve Deferans (Kod Analizi)

Adres okuma ve adresteki veriyi işaretçi üzerinden güncelleme örneği:

```cpp
#include <iostream>

int main() {
    int sensorVerisi = 100;
    int* ptr = &sensorVerisi; // Adresi işaretçiye ata

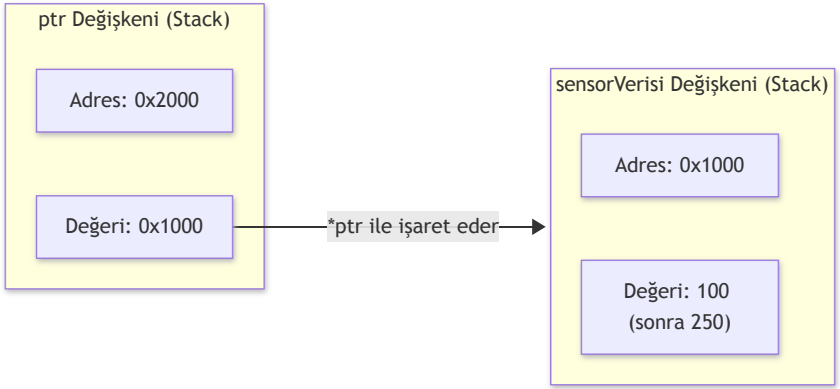
    std::cout << "sensorVerisi Değeri : " << sensorVerisi << "\n";
    std::cout << "sensorVerisi Adresi (&): " << &sensorVerisi << "\n";
    std::cout << "ptr Değeri (Adres)   : " << ptr << "\n";
    std::cout << "ptr Deferans (*ptr)  : " << *ptr << "\n";

    *ptr = 250; // Deferans ile adresteki veriyi doğrudan değiştir
    std::cout << "Güncel sensorVerisi  : " << sensorVerisi << "\n";
    return 0;
}
```

```
sensorVerisi Değeri : 100 | ptr Değeri : 0x7ffee4ba5c08 | *ptr : 100 | Güncel : 250
```

# İşaretçi ve Bellek İlişkisi

İşaretçi de kendi adresine sahip bağımsız bir değişkendir:




- `ptr` değişkeninin bellekteki kendi adresi `0x2000` 'dir.
- `ptr` değişkeninin içinde saklanan değer ise `sensorVerisi` 'nin adresi olan `0x1000` 'dir.
- `*ptr` işlemi yapıldığında CPU `0x1000` adresine gider ve o hücredeki değeri işler.

# İlklendirilmemiş İşaretçi Tehlikesi ve nullptr

Başlangıç değeri verilmemiş işaretçiler rastgele bir bellek adresini gösterir (*Wild Pointer*):

```
1  int* tehlikeliPtr; // İlklendirilmemiş! Rastgele bir adresi gösterir
2  // *tehlikeliPtr = 50; // KRİTİK HATA: Bellek koruma hatası (Segmentation Fault)!
3
4  int* guvenliPtr = nullptr; // Güvenli ilklendirme: Hiçbir yeri göstermez
```

- Vahşi İşaretçi (Wild Pointer): Tanımlanıp adres atanmamış işaretçiyi deferans etmek işletim sisteminin korumalı belleğine yazmaya çalışabilir ve program anında çöker.
- Modern C++ Standardı ( `nullptr` ): C++11 ile gelen `nullptr` , tür güvenli bir boş işaretçi sabitidir.
- C-Tarzı `NULL` / `0` Riski: `NULL` makrosu tam sayı `0` ile örtüşebildiğinden fonksiyon aşırı yüklemelerinde tür karmaşasına yol açar; modern C++'ta daima `nullptr` tercih edilir.

**Segmentation Fault:** İşletim sisteminin izin vermediği bir bellek bölgesine erişim teşebbüsünde oluşan donanım kesmesi.

# nullptr Denetimi ve Güvenli Kullanım Kalıbı

İşaretçiler deferans edilmeden önce mutlaka geçerli bir adrese sahip olup olmadığı denetlenmelidir:

```
1  void sensorOku(const int* ptr) {
2      if (ptr == nullptr) { // veya doğrudan: if (!ptr)
3          std::cerr << "Hata: Geçersiz sensör adresi!\n";
4          return;
5      }
6      std::cout << "Okunan Telemetri: " << *ptr << "\n";
7  }
```

- `nullptr` olan bir işaretçi mantıksal koşulda `false`, geçerli bir adres tutan işaretçi ise `true` üretir.
- `nullptr` deferans edilirse ( `*ptr` ), CPU sıfır adresine erişmeye çalışır ve anında çökme yaşanır.
- **Savunmacı Programlama İlkesi:** Dışarıdan işaretçi alan her fonksiyon önce `nullptr` denetimi yapmalıdır.

# Sabitler ve Göstericiler (Pointers and Const)

`const` anahtar kelimesinin `*` işaretine göre konumu anlamı tamamen değiştirir:

- 1. Normal İşaretçi ( `int* ptr` ): Hem gösterilen adres hem de adresteki veri değiştirilebilir.
- 2. Sabit Veriye İşaretçi ( `const int* ptr` veya `int const* ptr` ):
  - Gösterilen veri sabittir ( `*ptr = 20;` derleme hatası verir).
  - İşaretçinin tuttuğu adres değiştirilebilir ( `ptr = &b;` geçerlidir).
- 3. Sabit İşaretçi ( `int* const ptr` ):
  - İşaretçinin tuttuğu adres sabittir ( `ptr = &b;` derleme hatası verir).
  - Adresteki veri değiştirilebilir ( `*ptr = 20;` geçerlidir).
- 4. Sabit Veriye Sabit İşaretçi ( `const int* const ptr` ):
  - Ne adres ne de adresteki veri değiştirilebilir. Tamamen salt-okunurdur.

**Salt-Okunur (Read-Only):** Bellek içeriğinin yazmaya karşı kilitlendiği koruma modu.

# Sabitler ve Göstericiler: Kod ve Kural Tablosu

| Tanım Biçimi | Adres Değişebilir mi? | Veri Değişebilir mi? | Kullanım Amacı |
|---|---|---|---|
| `int* ptr` | Evet | Evet | Standart değişken işaretçisi |
| `const int* ptr` | Evet | ❌ Hayır | Salt-okunur tampon okuma |
| `int* const ptr` | ❌ Hayır | Evet | Sabit donanım register portu |
| `const int* const ptr` | ❌ Hayır | ❌ Hayır | Sabit kalibrasyon tablosu |

```
1  int a = 10, b = 20;
2  const int* p1 = &a; // Veri sabit
3  // *p1 = 15;        // HATA! Veri değiştirilemez.
4  p1 = &b;            // Geçerli: Başka adresi gösterebilir.
5
6  int* const p2 = &a; // Adres sabit
7  *p2 = 15;           // Geçerli: Veri güncellenebilir.
8  // p2 = &b;         // HATA! Adres değiştirilemez.
```

# Jenerik Göstericiler: void* Mimarisi

`void*`, herhangi bir veri tipine ait bellek adresini saklayabilen tip-bağımsız işaretçidir:

- **Jenerik Adres Temsili:** `int`, `double`, `char` veya kullanıcı tanımlı bir yapının adresi doğrudan `void*` işaretçisine atanabilir.
- **Doğrudan Deferans Yasağı:** Derleyici `void` tipinin bellekte kaç bayt kapladığını bilmediğinden `*voidPtr` doğrudan **okunamaz ve yazılamaz** (derleme hatası).
- **Aritmetik Yasağı:** Standart C++'ta `void*` üzerinde `ptr++` veya `ptr + 1` gibi işaretçi aritmetiği yapılamaz (adım büyüklüğü belirsizdir).
- **Tür Dönüşüm Zorunluluğu:** Değere erişmek için işaretçi asıl türüne dönüştürülmelidir (`static_cast`).

> **Mekatronik Kullanımı:** UART veya CAN Bus üzerinden gelen ham bayt paketleri jenerik `void*` tamponunda toplanır, ardından ilgili telemetri yapısına dönüştürülür.

# void* ve Tür Dönüşümü (static_cast) Kodu

Jenerik işaretçinin güvenli tür dönüşümü ile çözümlenmesi:

```cpp
#include <iostream>

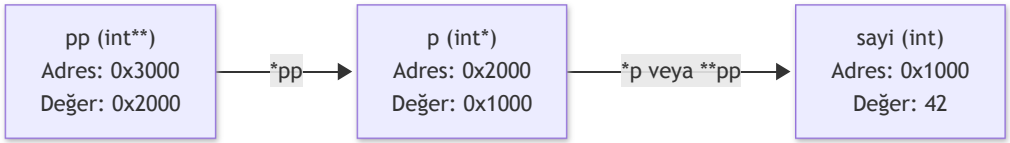
void veriYazdir(void* ptr, char tip) {
    if (ptr == nullptr) return;

    if (tip == 'i') {
        int* intPtr = static_cast<int*>(ptr); // Güvenli tür dönüşümü
        std::cout << "Tamsayı Değeri: " << *intPtr << "\n";
    } else if (tip == 'd') {
        double* dblPtr = static_cast<double*>(ptr);
        std::cout << "Kayan Nokta Değeri: " << *dblPtr << "\n";
    }
}

int main() {
    int sayi = 42;
    double gerilim = 3.3;
    veriYazdir(&sayi, 'i');
    veriYazdir(&gerilim, 'd');
```

# Göstericiye Gösterici (Pointer to Pointer - **)

Bir işaretçinin de bellekte bir adresi vardır; bu adresi tutan işaretçiye **işaretçiye işaretçi** denir:




- `p` , `sayi` değişkeninin adresini ( `&sayi` ) tutar.
- `pp` , `p` işaretçisinin adresini ( `&p` ) tutar.
- `*pp` , `p` işaretçisinin değerini ( `&sayi` yani `0x1000` ) verir.
- `**pp` , iki kademeli deferans yaparak asıl `sayi` değerine ( `42` ) ulaşır.

**Katmanlı Erişim:** Çok boyutlu dinamik diziler ve fonksiyon içinde dışarıdaki bir işaretçiyi yeniden yönlendirmek için kullanılır.

# Göstericiye Gösterici: Kod Örneği

İki kademeli adresleme mekanizmasının kod üzerindeki doğrulaması:

```cpp
#include <iostream>

int main() {
    int hedefHiz = 1500;    // RPM
    int* ptr = &hedefHiz;   // hedefHiz adresini tutan işaretçi
    int** ptrPtr = &ptr;    // ptr işaretçisinin adresini tutan işaretçi

    std::cout << "hedefHiz Değeri : " << hedefHiz << "\n";
    std::cout << "*ptr Değeri     : " << *ptr << "\n";
    std::cout << "**ptrPtr Değeri  : " << **ptrPtr << "\n";

    // İki kademe deferans ile orijinal değeri güncelleme
    **ptrPtr = 1800;
    std::cout << "Güncel hedefHiz  : " << hedefHiz << " RPM\n";
    return 0;
}
```

```
hedefHiz Değeri : 1500 | *ptr : 1500 | **ptrPtr : 1500 | Güncel hedefHiz : 1800 RPM
```

# Bölüm 1 Değerlendirmesi: İşaretçi Temelleri

Şu ana kadar ele aldığımız temel işaretçi mekanizmaları:

- Adres ve Deferans: `&` adresi alır, `*` adresteki veriye erişir ve günceller.
- `nullptr` Güvenliği: Boş işaretçileri sıfırlamak ve deferans öncesi kontrol etmek çökmeleri önler.
- Sabitlik Sözleşmesi: `const int*` veriyi kilitler, `int* const` adresi kilitler.
- `void*` Jenerikliği: Tip-bağımsız adresleme sağlar; `static_cast` ile dönüştürülmelidir.
- `**` Çift Gösterici: Bir işaretçinin adresini saklayarak çok boyutlu yapılara kapı açar.

> Sonraki Bölüm: İşaretçi Aritmetiği, Dizilerle İlişkisi ve Fonksiyonlara Parametre Aktarımı.

# İşaretçi Aritmetiği: Adres Matematiği

İşaretçiler üzerinde tamsayı ekleme ( `+` ), çıkarma ( `-` ), artırma ( `++` ) ve azaltma ( `--` ) yapılabilir:

- **Tip Boyutuna Göre İlerleme Kuralı:** İşaretçiyi 1 artırmak ( `ptr++` ), adresi 1 bayt değil, işaret edilen veri tipinin boyutu ( `sizeof(T)` ) kadar ileri taşır.
- Adres Hesaplama Formülü:

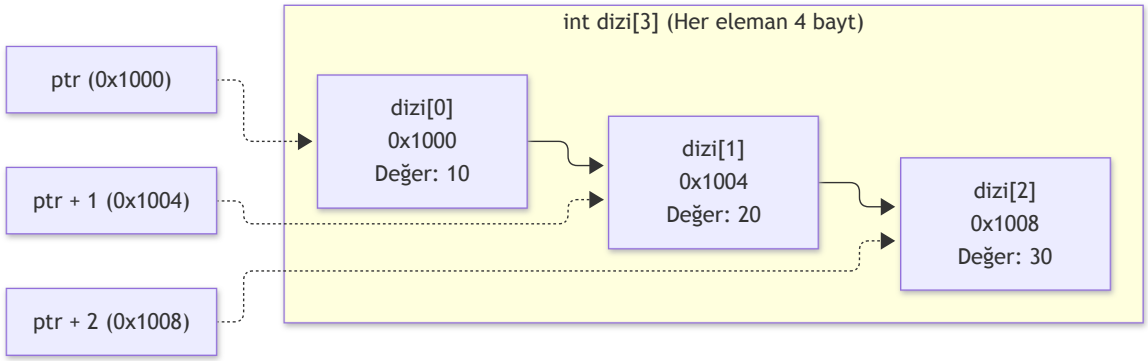
$$\text{Yeni Adres} = \text{Mevcut Adres} \pm (n \times \text{sizeof}(T))$$

- Örnek: `int* ptr` adresi `0x1000` ise ( `sizeof(int) = 4` bayt):
  - `ptr + 1` $\longrightarrow$ `0x1004`
  - `ptr + 2` $\longrightarrow$ `0x1008`
- İki Gösterici Farkı ( `ptr2 - ptr1` ): İki işaretçi arasındaki bayt farkını değil, aradaki **eleman sayısını** verir ( `ptrdiff_t` türünde).

> **ptrdiff_t:** İki işaretçi arasındaki farkı temsil eden standart işaretli tamsayı türü.

# İşaretçi Aritmetiği: Bellek Modeli

Bellekte 4 baytlık tamsayılar üzerinde `ptr++` ve `ptr + 2` adımları:




Uygulama Notu: `double* ptr` olsaydı ( `sizeof(double) = 8` bayt), `ptr + 1` adresi 8 bayt ileri ( `0x1000` $\rightarrow$ `0x1008` ) öteleyecekti.

# Diziler ve İşaretçi Özdeşliği (Array Decaying)

C++'ta dizi adı, dizinin ilk elemanının bellek adresine işaret eden sabit bir işaretçi gibi davranır:

```
1  int sensorler[4] = {12, 45, 78, 90};
2  int* ptr = sensorler; // &sensorler[0] ile tamamen aynıdır!
```

- İndisleme ile Aritmetik Özdeşliği:

$$\text{sensorler}[i] \equiv *(\text{sensorler} + i) \equiv *(ptr + i) \equiv ptr[i]$$

- Dizi-İşaretçi Bozunması (Array Decaying): Bir dizi fonksiyona aktarıldığında veya işaretçiye atandığında boyut bilgisini kaybeder ve ham bir işaretçiye ( `T*` ) dönüşür.
- Fark: Dizi adı sabit bir adrestir ( `sensorler++` derleme hatası verir), ancak işaretçi değişkendir ( `ptr++` geçerlidir).

> **Array Decaying:** Bir dizinin fonksiyona geçirilirken ilk elemanının işaretçisine indirgenmesi.

# Dizi ve İşaretçi Taraması (Kod Analizi)

İndisleme ve işaretçi aritmetiği ile bellek geziniminin karşılaştırılması:

```cpp
#include <iostream>

int main() {
    int adc[4] = {512, 620, 715, 804};
    int* ptr = adc; // İlk elemanın adresi

    std::cout << "1. İndisleme ile Erişim:\n";
    for (int i = 0; i < 4; ++i) {
        std::cout << "adc[" << i << "] = " << adc[i] << " (Adres: " << &adc[i] << ")\n";
    }

    std::cout << "\n2. İşaretçi Aritmetiği ile Erişim:\n";
    for (int i = 0; i < 4; ++i) {
        std::cout << "*(ptr + " << i << ") = " << *(ptr + i)
                  << " (Adres: " << (ptr + i) << ")\n";
    }
    return 0;
}
```

# Fonksiyon Parametre Aktarımı: 3 Yöntem

C++ dilinde fonksiyonlara argüman göndermenin 3 temel yaklaşımı:

- 1. Değer ile Aktarım (Pass-by-Value):
  - Değişkenin kopyası oluşturulur. Fonksiyon içindeki değişiklikler orijinal veriyi etkilemez.
  - Büyük veri yapılarında bellek kopyalama maliyeti ($\mathcal{O}(n)$) oluşturur.
- 2. Gösterici ile Aktarım (Pass-by-Pointer):
  - Değişkenin bellek adresi aktarılır. Fonksiyon orijinal veriyi doğrudan güncelleyebilir.
  - `nullptr` kontrolü gerektirir; isteğe bağlı (*optional*) parametreler için uygundur.
- 3. Referans ile Aktarım (Pass-by-Reference):
  - Değişkenin takma adı (*alias*) aktarılır. Orijinal veriyi doğrudan günceller.
  - `nullptr` olamaz, sözdizimi temizdir. Salt-okunur durumlarda `const T&` tercih edilir.

# Parametre Aktarımı: Kod Karşılaştırması

Değişken güncelleme yöntemlerinin karşılaştırmalı kod analizi:

```cpp
#include <iostream>

void degerleArtir(int x)        { x += 10; } // Sadece yerel kopya artar
void isaretciyleArtir(int* x)   { if (x) *x += 10; } // Orijinal veri artar
void referanslaArtir(int& x)    { x += 10; } // Orijinal veri artar

int main() {
    int sayi = 100;

    degerleArtir(sayi);
    std::cout << "Değer Sonrası   : " << sayi << "\n"; // 100 (Değişmedi)

    isaretciyleArtir(&sayi);
    std::cout << "İşaretçi Sonrası: " << sayi << "\n"; // 110 (Değişti)

    referanslaArtir(sayi);
    std::cout << "Referans Sonrası: " << sayi << "\n"; // 120 (Değişti)
    return 0;
}
```

# C++ Bellek Mimarisi: 4 Temel Bölge

Bir C++ programı çalıştırıldığında işletim sistemi tarafından RAM 4 ana bölgeye ayrılır:

- 1. Kod / Metin Segmenti (Code/Text Segment):
  - Derlenmiş makine kodları ve program komutları yer alır. Salt-okunurdur.
- 2. Veri Segmenti (Data & BSS Segments):
  - Programın yaşam döngüsü boyunca yaşayan `global` ve `static` değişkenler saklanır.
- 3. Yığın (Stack):
  - Fonksiyon çağrı çerçeveleri, yerel değişkenler ve parametreler saklanır. Otomatik yönetilir (LIFO).
- 4. Öbek / Serbest Bellek (Heap / Free Store):
  - Çalışma zamanında programcının `new` ile talep ettiği dinamik bellek alanıdır. Manuel yönetilir.

**BSS Segment:** Başlangıç değeri atanmamış statik ve global değişkenlerin sıfırlandığı bellek bölgesi.

# Bellek Anatomisi: Stack vs Heap

RAM içerisindeki mantıksal bellek büyüme yönleri ve organizasyonu:

# Yığın (Stack) ve Öbek (Heap) Karşılaştırması

İki bellek bölgesinin mimari ve performans özellikleri:

| Karşılaştırma Kriteri | Yığın (Stack) | Öbek / Serbest Bellek (Heap) |
|---|---|---|
| Yönetim Biçimi | Otomatik (Derleyici / CPU) | Manuel (Programcı: `new` / `delete`) |
| Erişim ve Tahsis Hızı | Son derece hızlı (CPU stack pointer) | Görece yavaş (İşletim sistemi arama yapar) |
| Boyut Sınırı | Kısıtlı (Genellikle 1 - 8 MB) | Geniş (Sistemdeki kullanılabilir tüm RAM) |
| Ömür Süresi | Tanımlandığı kapsam ( `{}` ) bitince | `delete` çağrılana veya program bitene kadar |
| Tahsis Zamanı | Derleme zamanında boyutu bilinir | Çalışma zamanında dinamik boyutlandırılır |
| Temel Hata Riski | Stack Overflow (Taşma) | Bellek sızıntısı (*Memory Leak*), Fragmantasyon |

# C vs C++ Dinamik Bellek Yönetimi

C ve C++ dinamik bellek ayırma yöntemleri arasındaki kritik farklar:

- C Dili ( `malloc` / `free` ):
  - Kütüphane fonksiyonlarıdır ( `<cstdlib>` ).
  - `void*` döndürür; açık tür dönüşümü (*type cast*) gerektirir.
  - İstenen bellek miktarını bayt cinsinden ( `sizeof` ) hesaplamayı zorunlu kılar.
  - Nesne kurucularını (constructor) ve yıkıcılarını (destructor) ASLA çağırmaz!
- C++ Dili ( `new` / `delete` ):
  - Dilin çekirdek operatörleridir.
  - İlgili tipin göstericisini doğrudan döndürür (Tip güvenlidir).
  - Boyutu derleyici otomatik hesaplar ( `sizeof` gerekmez).
  - Sınıf nesnelerinin kurucu ve yıkıcı metotlarını otomatik çalıştırır.

> **Tasarım Notu:** C++ nesne yönelimli projelerde `malloc/free` kesinlikle kullanılmamalıdır; nesne yaşam döngüsünü bozar.

# Tek Değişken İçin Dinamik Bellek: new ve delete

Heap üzerinde tek bir değişken oluşturma ve bellekten temizleme:

```cpp
#include <iostream>

int main() {
    // 1. Heap üzerinde dinamik int tahsisi ve ilk değer atama (100)
    int* ptr = new int(100);

    std::cout << "Heap Adresi : " << ptr << "\n";
    std::cout << "Heap Değeri : " << *ptr << "\n";

    // 2. Belleği serbest bırakma
    delete ptr;

    // 3. Sarkan göstericiyi (dangling pointer) engelleme
    ptr = nullptr;

    return 0;
}
```

- `new int(100)` : Heap'ten 4 bayt ayırır, içine 100 yazar ve adresi `ptr` 'ye verir.

# Dinamik Diziler: new ve delete

Çalışma zamanında boyutu belirlenen dizilerin Heap üzerinde tahsisi:

```cpp
#include <iostream>

int main() {
    int kanalSayisi = 0;
    std::cout << "Sensör kanal sayısını giriniz: ";
    std::cin >> kanalSayisi;

    // Heap üzerinde dinamik dizi tahsisi
    double* telemetri = new double[kanalSayisi];

    for (int i = 0; i < kanalSayisi; ++i) {
        telemetri[i] = (i + 1) * 1.25; // Veri atama
    }

    // Dizi belleğini serbest bırakma (KÖŞELİ PARANTEZ ZORUNLU!)
    delete[] telemetri;
    telemetri = nullptr;

    return 0;
```

**Kritik Kural:** `new[]` ile tahsis edilen dinamik diziler MUTLAKA `delete[]` ile serbest bırakılmalıdır! `delete telemetri` yazmak

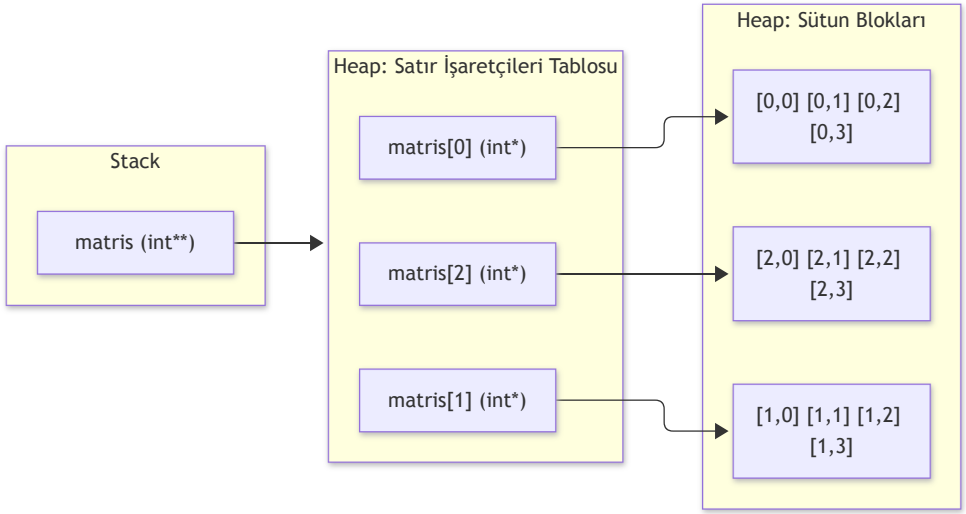
# İki Boyutlu Dinamik Dizi (Matris) Yönetimi

Boyutları çalışma zamanında belirlenen matrisler işaretçi dizisi ( `int**` ) ile kurulur:

```
1  int satir = 3, sutun = 4;
2
3  // 1. Adım: Satır işaretçilerini tutan ana dizi tahsisi
4  int** matris = new int*[satir];
5
6  // 2. Adım: Her satır için sütun dizilerinin tahsisi
7  for (int i = 0; i < satir; ++i) {
8      matris[i] = new int[sutun];
9  }
10
11 // Matris elemanına standart erişim: matris[i][j] = 0;
12
13 // 3. Adım: Belleği TEMİZLEME (Ters sıra ile yapılmalıdır!)
14 for (int i = 0; i < satir; ++i) {
15     delete[] matris[i]; // Önce her bir satır dizisi silinir
16 }
17 delete[] matris;        // Son olarak ana işaretçi dizisi silinir
18 matris = nullptr;
```

# İki Boyutlu Dinamik Matris Bellek Modeli

`int** matris` yapısının Heap üzerindeki blok organizasyonu:




> **Temizleme Kuralı:** Ana tablo önce silinirse sütun dizilerinin bellek adresleri kaybolur ve geri getirilemez bellek sızıntısı oluşur.

# Kritik Bellek Hatası 1: Bellek Sızıntısı (Memory Leak)

Dinamik olarak tahsis edilen belleğin `delete` edilmeden işaretçisinin kaybedilmesidir:

```
1  void telemetriIsle() {
2      int* tampon = new int[1000]; // Heap'ten 4000 bayt ayrıldı
3      // Hesaplamalar yapılır...
4      // delete[] tampon; UNUTULDU!
5  } // tampon işaretçisi Stack'ten silindi, ancak Heap'teki 4000 bayt sahipsiz kaldı!
```

- **Gömülü Sistem Tehlikesi:** Döngü içerisinde her saniye çağrılan bu fonksiyon kısa sürede mikrodenetleyicinin tüm RAM'ini tüketir.
- **Sonuç:** Yeni bellek tahsisleri başarısız olur ( `std::bad_alloc` ) ve otonom sistem aniden kilitlenir.
- **Çözüm Kuralı:** Her `new` çağrısına karşılık tam olarak bir `delete` çalışmalıdır.

> **Memory Leak:** Programın kullanmadığı halde işletim sistemine iade etmediği sahipsiz bellek blokları.

# Kritik Bellek Hatası 2: Sarkan Gösterici (Dangling Pointer)

Serbest bırakılmış (silinmiş) bir bellek adresini göstermeye devam eden işaretçidir:

```
1  int* ptr = new int(42);
2  delete ptr; // Bellek serbest bırakıldı, ancak ptr hala o adresi tutuyor!
3
4  // *ptr = 99; // KRİTİK HATA: Sarkan göstericiye yazma! Tanımsız Davranış!
```

- **Tehlike:** İşletim sistemi bu bellek alanını başka bir değişkene tahsis etmiş olabilir; sarkan gösterici üzerinden yazmak diğer veriyi sessizce bozar.
- **Çözüm (Güvenli Silme Deseni):** Bellek silindiği an işaretçiye hemen `nullptr` atanmalıdır:

```
1  delete ptr;
2  ptr = nullptr; // Artık sarkan gösterici değil, güvenli boş işaretçi
```

- `nullptr` olan bir işaretçiyi kazara silmek güvenlidir ( `delete nullptr;` hiçbir işlem yapmaz).

# Kritik Bellek Hatası 3: Çift Serbest Bırakma (Double Free)

Aynı bellek adresinin birden fazla kez serbest bırakılmaya çalışılması durumudur:

```
1  int* ptr1 = new int(10);
2  int* ptr2 = ptr1; // Aynı Heap adresini paylaşan iki işaretçi
3
4  delete ptr1; // Bellek başarıyla serbest bırakıldı
5  delete ptr2; // KRİTİK HATA: Çift silme! Heap yönetim yapısı çöker!
```

- **Çökme Sebebi:** İşletim sisteminin Heap bellek yöneticisi serbest blok tablosunu güncellerken aynı adresi tekrar iade almaya çalışır ve güvenlik koruması devreye girerek programı sonlandırır.
- **Sığ Kopyalama Riski:** İlerleyen haftalarda sınıflarda göreceğimiz sığ kopyalama (*shallow copy*) bu hatanın en temel kaynağıdır.
- **Önlem:** Bellek sahipliği tek bir sorumluya verilmeli veya işaretçiler silindikten sonra `nullptr` yapılmalıdır.

# C++ Determinizmi ve RAII Mantığına Giriş

C++ neden Java veya C# dillerindeki gibi otomatik Çöp Toplayıcı (Garbage Collector) kullanmaz?

- **Determinizm İhtiyacı:** Robotik ve gömülü sistemlerde bir kaynağın veya motorun **tam olarak ne zaman** durdurulup temizleneceğinin mikro-saniye hassasiyetinde bilinmesi şarttır.
- **GC Gecikmeleri (Stop-the-World):** Çöp toplayıcılar periyodik olarak tüm programı durdurup RAM taraması yapar; bu duraksamalar gerçek zamanlı kontrol döngülerini bozar.
- **Modern C++ Çözümü (RAII):** *Resource Acquisition Is Initialization*
  - Kaynak (bellek, dosya, soket) kurucu fonksiyonda (*constructor*) alınır.
  - Kapsamdan çıkıldığında yıkıcı fonksiyon (*destructor*) tarafından otomatik `delete` edilir.
  - Gelecek haftalarda göreceğimiz Akıllı İşaretçilerin ( `std::unique_ptr` ) temel felsefesidir.

**RAII:** Kaynak tahsisinin nesne ömrüne bağlanarak bellek sızıntısını sıfıra indiren temel C++ tasarım ilkesi.